Travel Without Barriers

Technical Report

This report outlines the key design and architectural choices behind **Travel Without Barriers**, a deployed full-stack REST API and web application that recommends lower-friction travel destinations and supports user-managed wishlists. This report aims to justify why each major technical choice was made, what trade-offs were accepted, what challenges emerged during implementation, how the system was tested, and how Generative AI was used critically.

Project Scope and Objectives

- **Primary aim:** help users facing financial or mental-health-related barriers to travel identify destinations that feel realistic, affordable, less crowded, and easier to plan.
- **System scope:** recommendation API, authentication, CRUD wishlist management, deployed frontend, deployed backend, and a MCP layer for AI-accessible tooling.
- **Design goal:** keep the user experience calm and low-friction while maintaining a technically credible backend architecture that satisfies the coursework brief.
- **Output focus**: a working deployed service supported by documentation, testing evidence, and reflective analysis of design decisions.

Submission Links

- **GitHub repository:** https://github.com/sarahphiri/web_services_coursework
- **API documentation:** https://github.com/sarahphiri/web_services_coursework/blob/main/API_Documentation_TWB.pdf
- **Presentation slides and visuals:** https://leeds365-my.sharepoint.com/:p:/g/personal/sc22s2p_leeds_ac_uk/IQAc5FNWk0n8QKSVaA56aedaAU9DlkdSR8TrvXRsRaLmU4Y?e=ljX5lX

Please note: The powerpoint link is only accessible through a University of Leeds account.

High-Level Solution

- **Frontend:** Next.js interface on Vercel for destination discovery, filtering, user login, and wishlist interaction.
- **Backend:** FastAPI on Railway for REST endpoints, validation, authentication, recommendation logic, and persistence access.
- **Database:** SQLite used as the relational store for destinations, users, wishlists, and wishlist items.
- **AI interface:** MCP server implemented in Python and refactored to call the live API so AI-triggered actions affect the deployed system, not a local-only database.

Technology Stack and Architectural Justification

The technology stack was chosen to balance simplicity, development speed, and maintainability. The goal was to use tools that fit the project's requirements, supported rapid iteration during development, and could be easily deployed and tested.

Programming Language: Python

- Python allowed the project to use **one language for both API development and dataset preparation**, simplifying the workflow when importing and transforming tourism data.
- Its strong ecosystem for **data processing and scripting** made it easy to generate import scripts and experiment with recommendation metrics.
- **Python integrates naturally with FastAPI**, enabling rapid development while keeping the codebase readable and maintainable.

Backend Framework: FastAPI

- **FastAPI provides automatic OpenAPI documentation**, which directly supported the coursework requirement to clearly document all API endpoints.
- **Pydantic request validation** ensured that inputs such as authentication credentials and wishlist data were validated before reaching the database layer.
- The built-in **interactive /docs interface** allowed endpoints to be tested independently of the frontend, which was extremely useful during debugging and deployment.

Frontend Framework: Next.js

- **Next.js** was selected to create a polished frontend quickly while still supporting modular component-based development.
- Its routing model made it straightforward to build separate pages for recommendations and wishlists, while **React** state management supported responsive UI feedback such as login status, button state changes, and filter interactions.
- Deploying to **Vercel** aligned naturally with **Next.js** and reduced deployment complexity for the frontend layer.

Database Choice: SQLite

- **SQLite was chosen because the dataset (~2000 destinations) is small**, meaning a lightweight embedded database was sufficient without requiring a separate database server.
- It simplified development by allowing **fast local setup**, **easy schema inspection, and quick dataset imports**, which was useful while iterating on recommendation metrics.
- **A relational schema (users, wishlists, wishlist_items)** fits naturally with SQL relationships and constraints, making SQLite a better choice than a NoSQL alternative for this data model.

Architectural Shape

- **Layered architecture (client → API → logic → database)** was used to separate responsibilities. This made it easier to debug recommendation scoring, authentication, and frontend behaviour independently during development
- **The API acts as the single source of truth**, with both the web frontend and the MCP server interacting with the same deployed endpoints. This ensured consistent behaviour across interfaces and prevented data duplication.
- This structure allowed the recommendation system, authentication logic, and wishlist functionality to **evolve independently** while keeping the overall system simple enough for a coursework prototype.

Data, API Design, and Security Decisions

Dataset Choice and Justification

The project uses the **Kaggle** dataset *"Popular Tourist Destinations and Their Features"* because it closely matches the product goal: recommending destinations that are affordable, lower-pressure, and highly rated. The dataset includes synthetic attributes such as **destination name**, **country**, **continent**, **type**, **estimated cost**, **rating**, **annual visitor numbers**, and **best season**, which map directly to the recommendation requirements. **ChatGPT** was used to help identify and compare candidate datasets, but the final choice was made because this dataset provided the cleanest conceptual fit for the barrier-reduction theme. Although the dataset contains **synthetic** data, it was selected because its global scope and structured attributes enable the API to *demonstrate* a wide range of functionality.

- **Dataset source:** https://www.kaggle.com/datasets/cosmox23/popular-tourist-destinations-and-their- features
- **Reason for fit:** contains the exact fields needed to model affordability, crowding, quality, and planning convenience.
- **Licensing:** the dataset was treated as academically usable under the supplied MIT licence included in **Appendix B**.

API Design Choices

- A **RESTful resource-oriented design** was used because it aligns with web service best practices and simplifies documentation, testing, and client integration.
- Separate resources were defined for destinations, recommendations, wishlists, and wishlist items, which keeps responsibilities **clear** and prevents endpoints from becoming **overloaded**.
- Recommendation calculations were exposed through a dedicated /recommendations endpoint, ensuring that ranking logic is applied **consistently** for all clients rather than duplicated in the frontend.

Recommendation Logic

- Raw dataset fields such as cost, visitor numbers, and ratings were transformed into **derived recommendation metrics** to make destination suggestions easier for users to interpret.
- The scoring system was intentionally designed to be **transparent** and **explainable**, so users can understand why destinations are recommended rather than relying on opaque algorithms.
- The recommendation metrics combine affordability, crowd levels, and quality to support the project goal of identifying destinations that reduce common barriers to travel.

Affordability score = 1 / (avg_cost_usd + 1)

Quietness Score = 10 / (annual_visitors_m + 1)

Quality score = rating

Hidden Gem Score = Quality Score + Quietness Score

Barrier Score = Affordability Score + Quietness Score + Quality Score

Authentication and Security

- Authentication was implemented so that **wishlist operations become user-specific rather than anonymous**, enabling ownership checks and personalised travel planning.
- Input validation was added for **blank credentials, duplicate users, blank wishlist names, and malformed requests**, helping prevent invalid system state.
- **Parameterised SQL queries** were used to reduce SQL injection risk, and CORS configuration was restricted during deployment to allow safe communication between the frontend and backend.
- The system improves security compared to a fully open prototype, although future versions should implement **fully signed JWT authentication and consistent password hashing across the entire stack**.

Deployment, Challenges, Testing, and Lessons Learned

Deployment Strategy

- **The frontend was deployed on Vercel**, which integrates directly with Next.js and automatically handles production builds, HTTPS, and continuous deployment from GitHub.
- **The backend API was deployed on Railway**, allowing the FastAPI service to be launched using a simple Uvicorn start command while keeping infrastructure configuration minimal.
- Environment variables were introduced so the frontend could communicate with the deployed backend rather than a local development URL, reflecting a realistic **separation between client and server environments**

Implementation Challenges

- Early deployment attempts exposed **schema drift in the SQLite database**. During development the database had been created locally several times while the schema was evolving, meaning older tables sometimes lacked newly added columns or default values. This caused **runtime errors** when queries expected fields that were not present. The issue was resolved by **reinitialising the schema** and ensuring the dataset import script **rebuilt** the database consistently.
- Differences between local and deployed environments revealed **hidden configuration assumptions**. Issues included stale browser authentication tokens after backend changes, missing start commands during the first Railway deployment, and CORS restrictions that initially prevented the Vercel frontend from communicating with the API. Resolving these required explicitly configuring environment variables, deployment start commands, and allowed origins.
- An architectural issue emerged during MCP development. The first MCP implementation interacted with a **local SQLite database**, while the deployed web application used the Railway-hosted backend. As a result, actions performed through MCP were not visible in the live frontend. The MCP server was refactored to call the deployed API instead, ensuring the API remained the **single source of truth** for all clients.

Lessons Learned

- Deployment revealed more meaningful engineering lessons than local-only development, particularly around **environment configuration, authentication handling, and cross-service communication**. Issues such as incorrect API URLs, stale authentication tokens, missing deployment start commands, and CORS restrictions only became visible once the frontend and backend were running in separate environments. Addressing these problems required explicitly configuring environment variables and deployment settings so that the system behaved consistently across development and production environments.
- Treating the API as a **product that others must understand and use** significantly improved documentation quality and endpoint clarity. Once the system needed to be usable by external clients (including the frontend, API documentation users, and MCP tools), clearer route naming, parameter validation, and detailed API documentation became essential.
- The project benefited from an **incremental development approach:** implementing basic routes first, then adding validation, deployment, and finally MCP integration. Building functionality in stages made it easier to isolate and debug issues because each new feature was layered on top of an already functioning system. This reduced complexity and made debugging significantly easier.
- These lessons will all be taken into consideration when developing projects in the future.

Testing Approach

- API endpoints were tested using **FastAPI's interactive Swagger documentation** (/docs), which allowed requests to be executed independently of the frontend during development.
- Positive-path testing verified core functionality including **destination recommendations, filtering, user registration, login, wishlist creation, deletion, and adding/removing items**.
- Negative-path testing included duplicate **user registration, blank credential validation, invalid authentication tokens, and empty wishlist names.**
- Deployment testing ensured that the **Vercel frontend successfully communicated with the Railway backend,** confirming that environment variables and CORS configuration were correctly aligned.
- MCP functionality was tested using the **MCP Inspector**, verifying that tool-triggered actions produced visible changes in the live deployed system.

Limitations, Future Development, and Generative AI Analysis

Current Limitations

- SQLite was selected for simplicity and ease of development, but it is not designed for **high-concurrency production workloads**. While it works well for a coursework prototype with a small dataset, a production deployment would require a **more scalable** database system.
- Authentication is functional but not yet **fully standardised**. Although protected routes and token validation are implemented, future versions should use fully signed JWT tokens and hashed password storage across the entire authentication workflow.
- The recommendation scoring system is deliberately transparent and interpretable rather than personalised. Destinations are ranked using cost, visitor numbers, and ratings, but the system does not currently incorporate **user preferences** or **behavioural feedback**.

Future Development

- Migrate persistence from SQLite to **PostgreSQL** or another production-grade relational database to support stronger durability, improved concurrency handling, and better scalability for multi-user environments.
- Strengthen security through **rate limiting**, **improved session handling**, and a **fully standardised JWT authentication flow** with consistently hashed password storage.
- Expand the MCP layer with **additional tools**, such as itinerary creation and richer travel-planning interactions, allowing AI clients to interact with more of the system's functionality.
- Improve reliability by adding **automated testing**, including unit tests for recommendation scoring logic and integration tests that verify endpoint behaviour across the full API.

Generative AI Declaration and Analysis

Generative AI was used extensively, but deliberately, across the lifecycle of the project. **ChatGPT** was the primary tool and **Gemini** was used as a secondary tool for comparative judgement and design generation. AI was used as a **code generator**, **design assistant**, **debugger**, **prompt-engineering partner**, **dataset discovery aid**, **documentation assistant and generator**, **deployment troubleshooter**, and **branding collaborator**. All generated suggestions were **reviewed**, **adapted**, and **tested** before being incorporated.

- **ChatGPT** example use cases: idea generation support *(including creating the overall concept idea for 'Travel Without Barriers')*, prompt generation, dataset discovery, code generation, recommendation-metric design, architecture alternatives, database schema ideas, import-script generation, backend route implementation, debugging, deployment issues, frontend fixes, MCP implementation, testing artefacts, documentation drafting, and reflective write-up support.

- **Gemini** example use cases: producing a branded visual identity and logo direction, plus offering a secondary perspective on documentation and design presentation.

• **Real prompt example used during development:** *"Can you write me a prompt to give to ChatGPT which would do this: (iii) Generate Import Scripts: Provide a subset of your dataset to AI and ask it to help you create a script to import and store the data in your SQL database. This can significantly accelerate the data preparation process."*

- The key justification for AI usage is that it accelerated exploration and comparison, not just implementation: it helped evaluate datasets, design metrics, compare options, and diagnose failures that arose duringdeployment and integration.

References and Required Citations

Kaggle dataset : https://www.kaggle.com/datasets/cosmox23/popular-tourist-destinations-and-their-features
GitHub repository: https://github.com/sarahphiri/web_services_coursework
Presentation and visuals: https://leeds365-my.sharepoint.com/:p:/g/personal/sc22s2p_leeds_ac_uk/IQAc5FNWk0n8QKSVaA56aedaAU9DlkdSR8TrvXRsRaLmU4Y?e=dHjefX
API documentation: https://github.com/sarahphiri/web_services_coursework/blob/main/API_Documentation_TWB.pdf

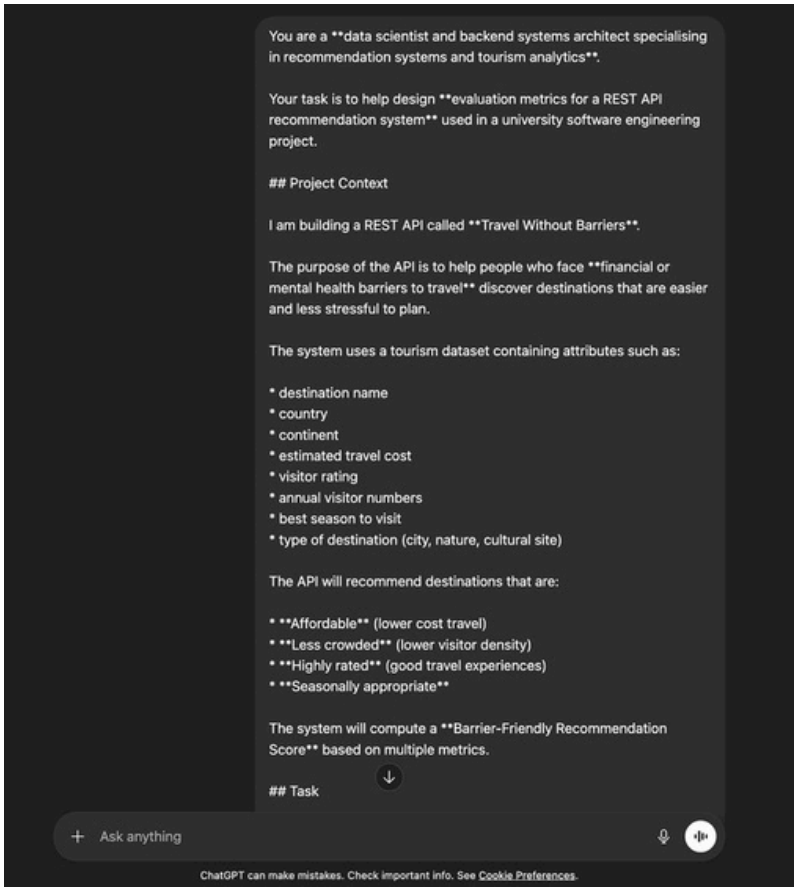
Appendix A — Generative AI Declaration, Analysis, and Conversation Logs

This appendix provides a fuller declaration of Generative AI usage together with supporting visual evidence. The screenshots included here are direct conversation logs from development sessions and illustrate how AI was used for different purposes, such as idea generation, problem framing, prompt engineering, dataset identification, import-script generation, debugging, architecture ideation, documentation support, branding, and implementation refinement. Related screenshots are identified as part of the same conversation thread where appropriate. Not **all** prompt conversations are included in this appendix, this appendix only features **key examples** of how GenAI has been used as evidence.

A.1 Detailed Declaration

- **Primary tool:** ChatGPT, used iteratively across planning, implementation, debugging, deployment, documentation, and reflective analysis.
- **Secondary tool:** Gemini, used to generate the branding and logo direction and to provide a second opinion on presentation/documentation framing.
- **Human role:** every AI-produced suggestion was reviewed, edited, tested, and integrated selectively; failures and inaccuracies were resolved through debugging and iteration.
- **Educational value:** GenAI was used not only for code generation but also for option evaluation, justification, architectural comparison, and the production of reusable prompts that accelerated later stages of the project.

Figure A1. Prompting ChatGPT to evaluate which recommendation metrics and formulas should be used to represent accessibility barriers such as affordability and crowd levels.



You are a **data scientist and backend systems architect specialising in recommendation systems and tourism analytics**.

Your task is to help design **evaluation metrics for a REST API recommendation system** used in a university software engineering project.

Project Context

I am building a REST API called **Travel Without Barriers**.

The purpose of the API is to help people who face **financial or mental health barriers to travel** discover destinations that are easier and less stressful to plan.

The system uses a tourism dataset containing attributes such as:

* destination name
* country
* continent
* estimated travel cost
* visitor rating
* annual visitor numbers
* best season to visit
* type of destination (city, nature, cultural site)

The API will recommend destinations that are:

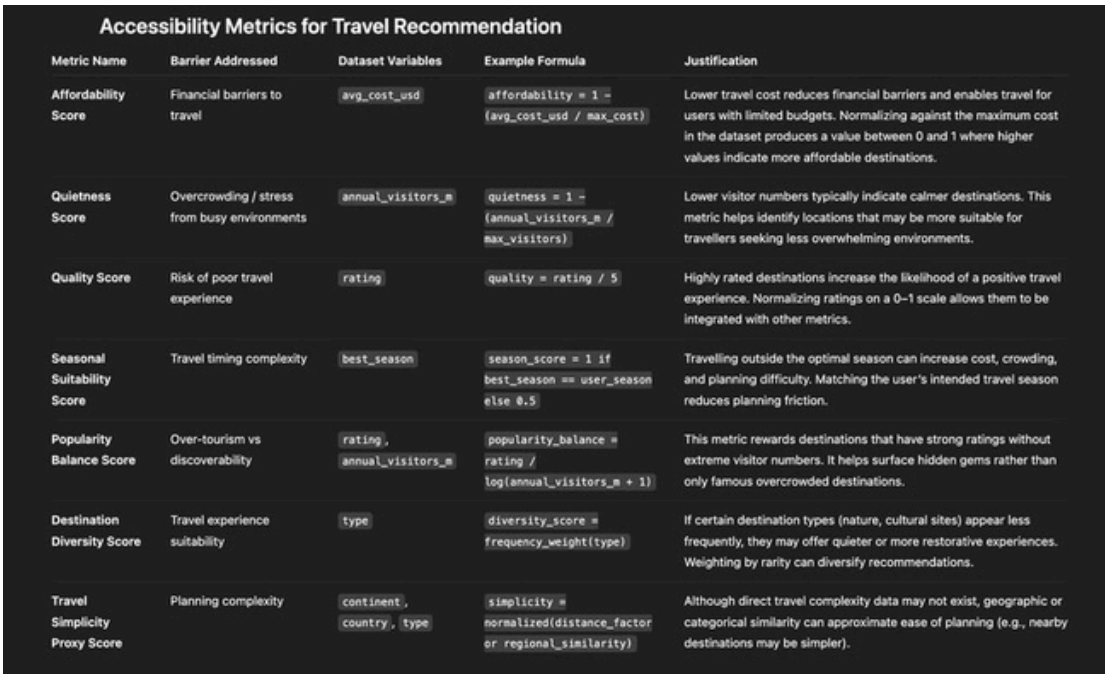
* **Affordable** (lower cost travel)
* **Less crowded** (lower visitor density)
* **Highly rated** (good travel experiences)
* **Seasonally appropriate**

The system will compute a **Barrier-Friendly Recommendation Score** based on multiple metrics.

Task

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

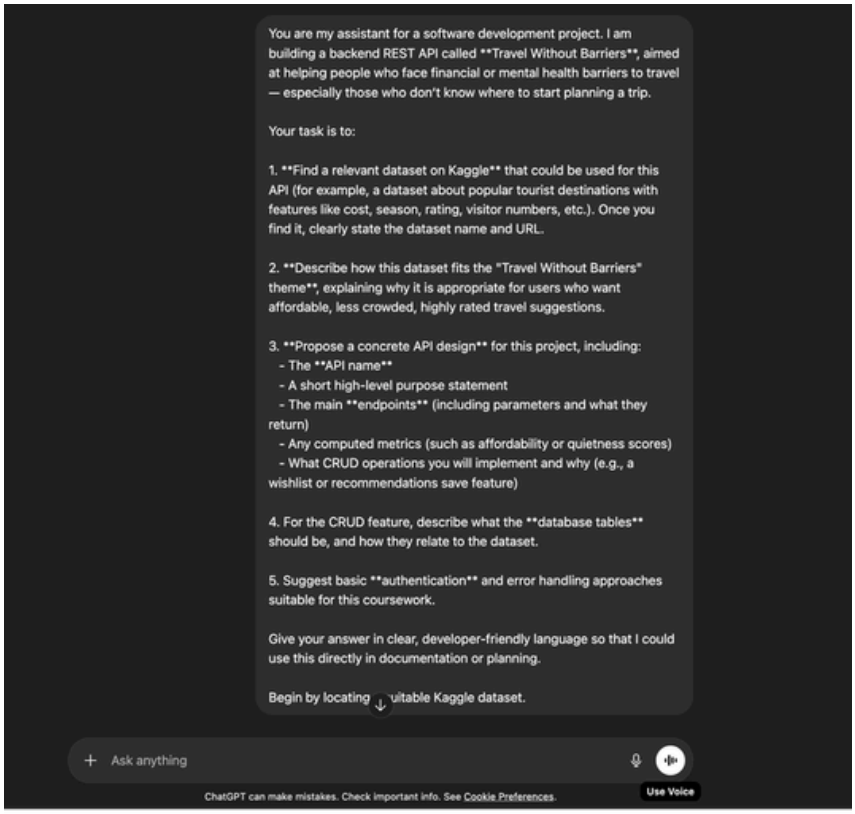


Accessibility Metrics for Travel Recommendation

Metric Name	Barrier Addressed	Dataset Variables	Example Formula	Justification
Affordability Score	Financial barriers to travel	`avg_cost_usd`	`affordability = 1 - (avg_cost_usd / max_cost)`	Lower travel cost reduces financial barriers and enables travel for users with limited budgets. Normalizing against the maximum cost in the dataset produces a value between 0 and 1 where higher values indicate more affordable destinations.
Quietness Score	Overcrowding / stress from busy environments	`annual_visitors_m`	`quietness = 1 - (annual_visitors_m / max_visitors)`	Lower visitor numbers typically indicate calmer destinations. This metric helps identify locations that may be more suitable for travellers seeking less overwhelming environments.
Quality Score	Risk of poor travel experience	`rating`	`quality = rating / 5`	Highly rated destinations increase the likelihood of a positive travel experience. Normalizing ratings on a 0–1 scale allows them to be integrated with other metrics.
Seasonal Suitability Score	Travel timing complexity	`best_season`	`season_score = 1 if best_season == user_season else 0.5`	Travelling outside the optimal season can increase cost, crowding, and planning difficulty. Matching the user's intended travel season reduces planning friction.
Popularity Balance Score	Over-tourism vs discoverability	`rating`, `annual_visitors_m`	`popularity_balance = rating / log(annual_visitors_m + 1)`	This metric rewards destinations that have strong ratings without extreme visitor numbers. It helps surface hidden gems rather than only famous overcrowded destinations.
Destination Diversity Score	Travel experience suitability	`type`	`diversity_score = frequency_weight(type)`	If certain destination types (nature, cultural sites) appear less frequently, they may offer quieter or more restorative experiences. Weighting by rarity can diversify recommendations.
Travel Simplicity Proxy Score	Planning complexity	`continent`, `country`, `type`	`simplicity = normalized(distance_factor or regional_similarity)`	Although direct travel complexity data may not exist, geographic or categorical similarity can approximate ease of planning (e.g., nearby destinations may be simpler).

The purpose was to obtain candidate evaluation metrics together with justifications, which directly informed the recommendation model. The overall decision remained a human one: the suggestions were reviewed and then translated into the project's affordability, quietness, hidden-gem, and barrier-score design. The prompt which was used was also generated by ChatGPT, as part of the prompt engineering use case.

Figure A2. Prompting ChatGPT to identify a suitable Kaggle dataset and propose an initial API design, endpoint structure, database tables, authentication approach, and error-handling strategy.



You are my assistant for a software development project. I am building a backend REST API called **Travel Without Barriers**, aimed at helping people who face financial or mental health barriers to travel — especially those who don't know where to start planning a trip.

Your task is to:

1. **Find a relevant dataset on Kaggle** that could be used for this API (for example, a dataset about popular tourist destinations with features like cost, season, rating, visitor numbers, etc.). Once you find it, clearly state the dataset name and URL.

2. **Describe how this dataset fits the "Travel Without Barriers" theme**, explaining why it is appropriate for users who want affordable, less crowded, highly rated travel suggestions.

3. **Propose a concrete API design** for this project, including:
   - The **API name**
   - A short high-level purpose statement
   - The main **endpoints** (including parameters and what they return)
   - Any computed metrics (such as affordability or quietness scores)
   - What CRUD operations you will implement and why (e.g., a wishlist or recommendations save feature)

4. For the CRUD feature, describe what the **database tables** should be, and how they relate to the dataset.

5. Suggest basic **authentication** and error handling approaches suitable for this coursework.

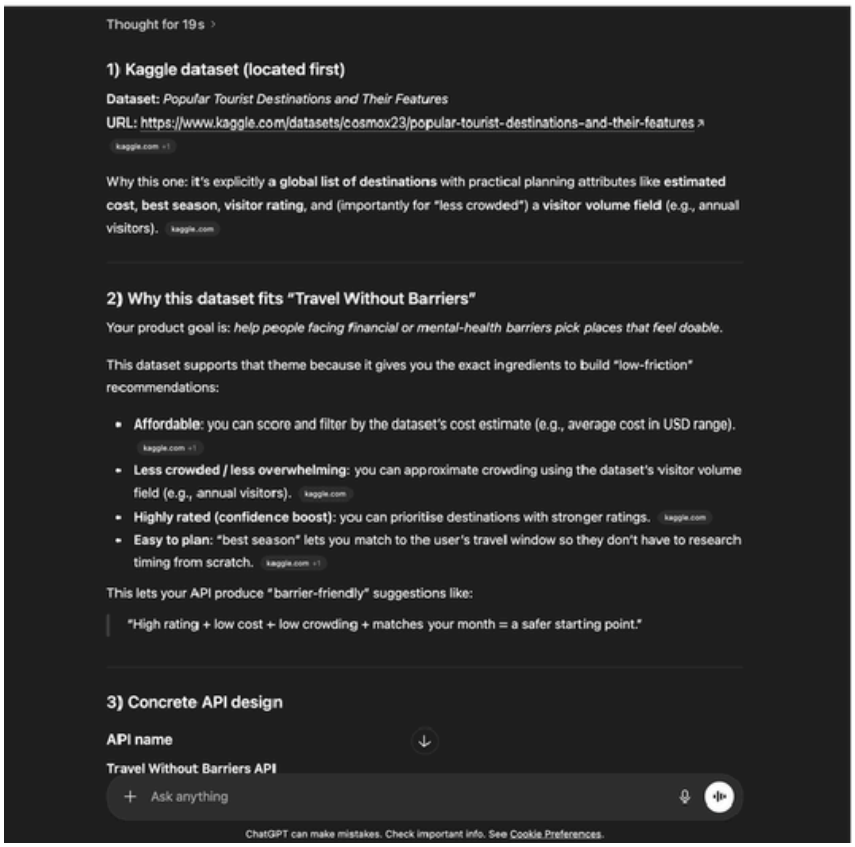
Give your answer in clear, developer-friendly language so that I could use this directly in documentation or planning.

Begin by locating a suitable Kaggle dataset.

Ask anything

Use Voice

ChatGPT can make mistakes. Check important info. See Cookie Preferences.



Thought for 19 s

1) Kaggle dataset (located first)

Dataset: *Popular Tourist Destinations and Their Features*

URL: https://www.kaggle.com/datasets/cosmox23/popular-tourist-destinations-and-their-features

Why this one: it's explicitly **a global list of destinations** with practical planning attributes like **estimated cost, best season, visitor rating**, and (importantly for "less crowded") a **visitor volume field** (e.g., annual visitors).

2) Why this dataset fits "Travel Without Barriers"

Your product goal is: *help people facing financial or mental-health barriers pick places that feel doable.*

This dataset supports that theme because it gives you the exact ingredients to build "low-friction" recommendations:

- **Affordable**: you can score and filter by the dataset's cost estimate (e.g., average cost in USD range).
- **Less crowded / less overwhelming**: you can approximate crowding using the dataset's visitor volume field (e.g., annual visitors).
- **Highly rated (confidence boost)**: you can prioritise destinations with stronger ratings.
- **Easy to plan**: "best season" lets you match to the user's travel window so they don't have to research timing from scratch.

This lets your API produce "barrier-friendly" suggestions like:

> "High rating + low cost + low crowding + matches your month = a safer starting point."

3) Concrete API design

API name

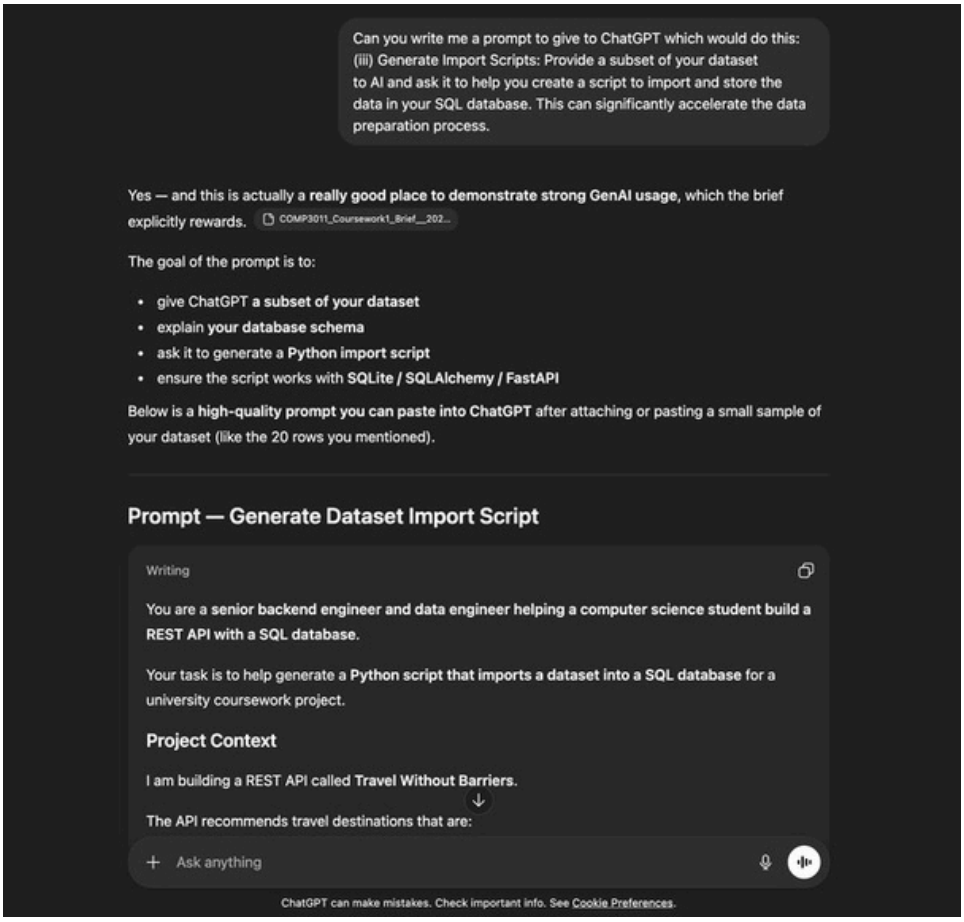
Travel Without Barriers API

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

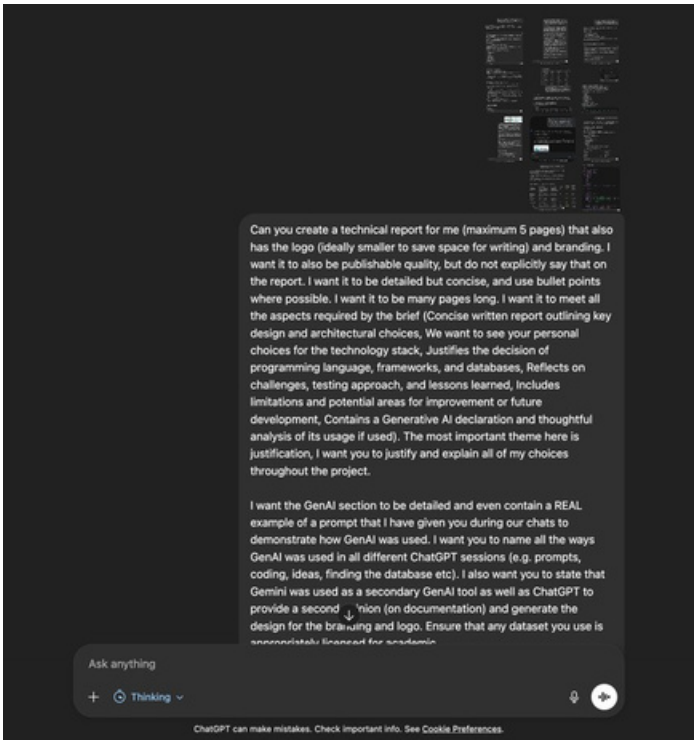
This screenshot is connected to Figure **A1** as part of the early design phase. It shows AI being used to accelerate structured planning: dataset evaluation, endpoint ideation, relational modelling, and architecture scoping. These suggestions were justified by AI, then finally agreed by me. This prompt is another example of a prompt that was generated by ChatGPT, as GenAI was used to generate effective prompts in some cases.

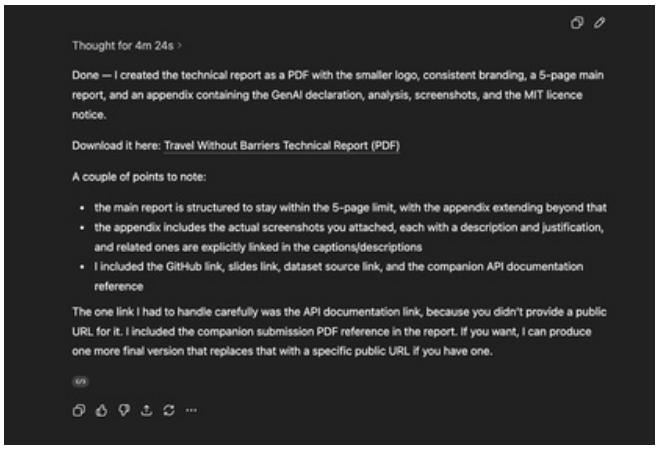
Figure A3. Prompt-engineering conversation focused on generating a reusable prompt for dataset import-script generation.

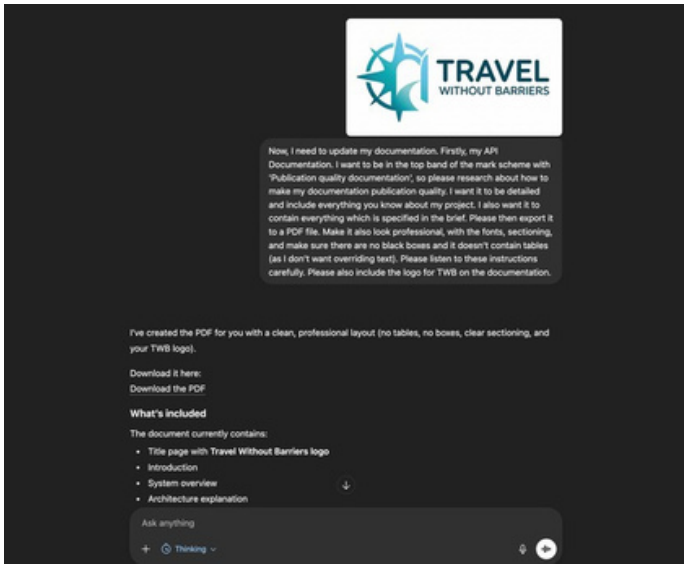


This shows how for some tasks I asked ChatGPT to help me create a prompt. In this case, it was to generate import scripts. This example shows AI being used recursively: ChatGPT was asked to produce a better prompt that could then be used in a dedicated coding session. This was valuable because it improved the quality of the downstream code-generation request and made the data-import phase faster and more structured. **A1** and **A2** also demonstrate the use of GenAI prompting. Not **all** prompts used during development were AI generated, as generated prompts were only used where I thought it was most suitable (for example, in cases where a lot of detail was needed).

Figure A4. Use of GenAI to create Technical Report and API Documentation

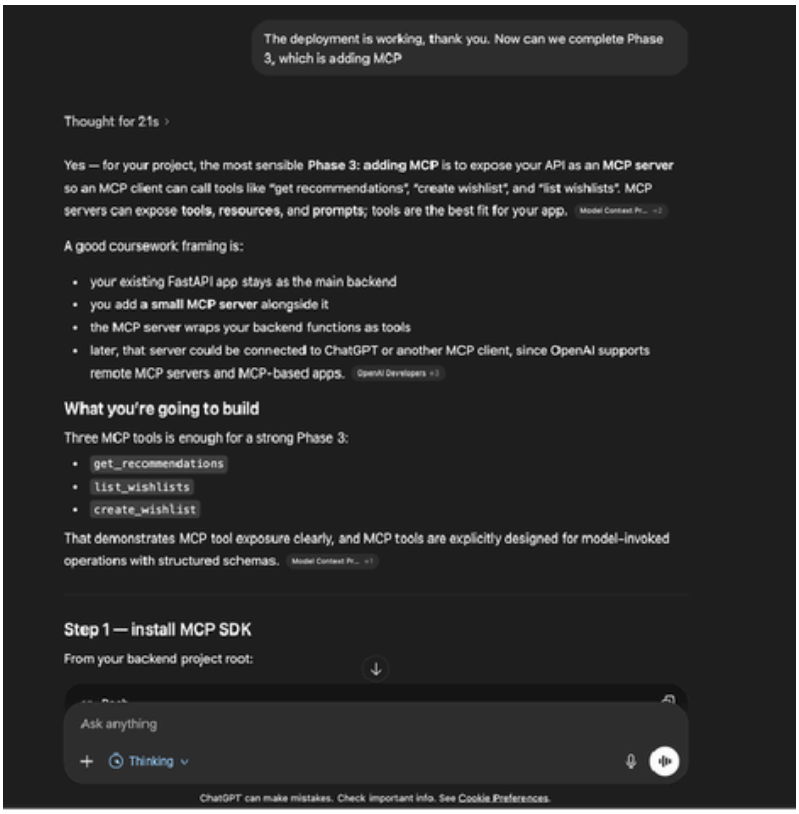


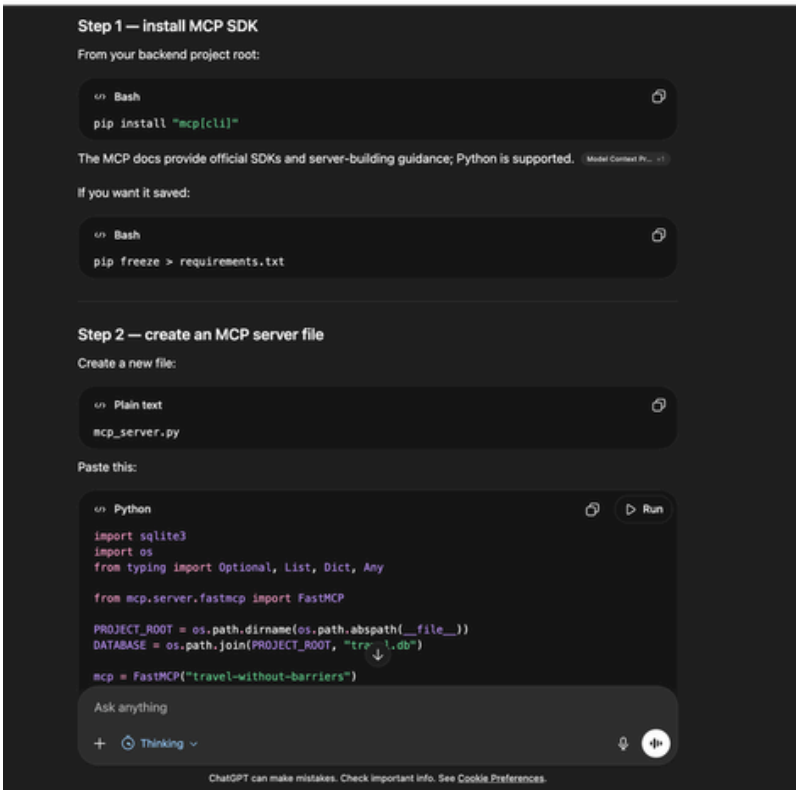




These screenshots demonstrate the use of GenAI to create my Technical Report and API Documenentation. They were both generated as PDFs by ChatGPT, using the branding of the project's frontend, and then added to and edited by me. GenAI was used to assist all versions of the reports, from version 1 to the final.

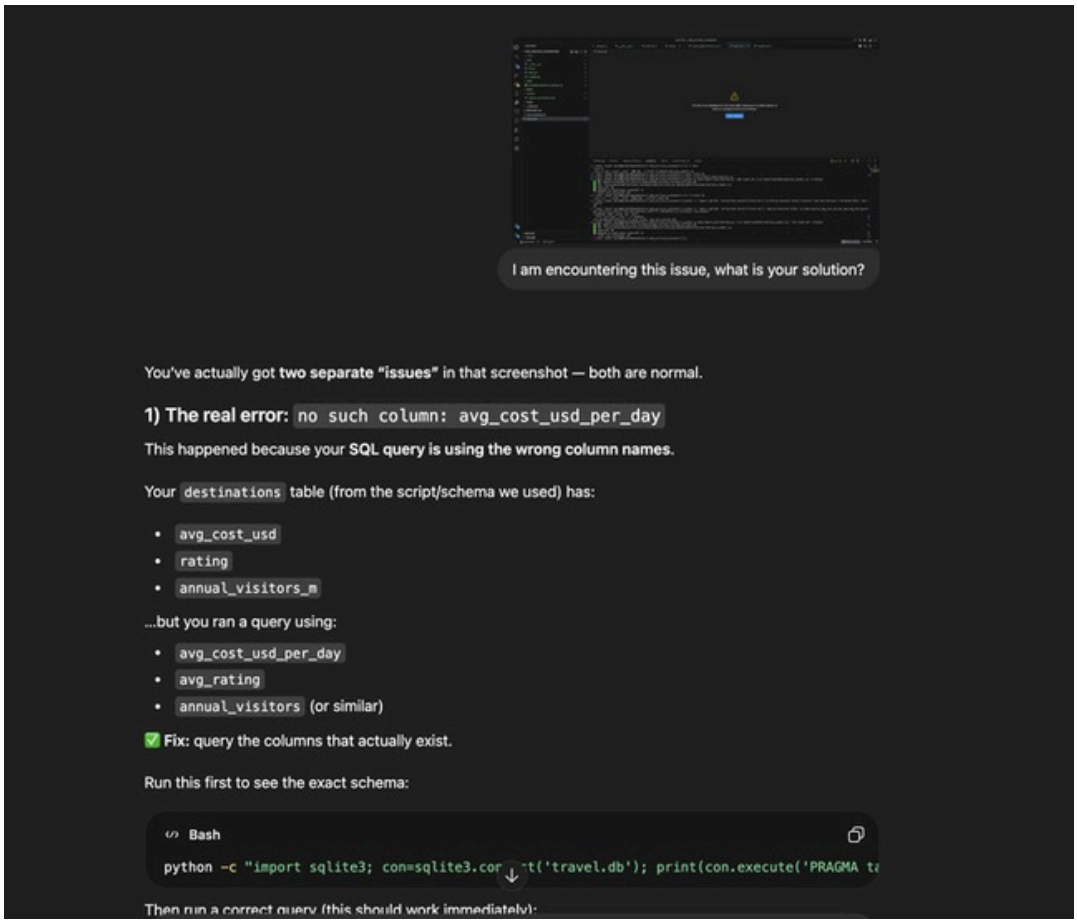
Figure A5. GenAI used for code generation and implementation





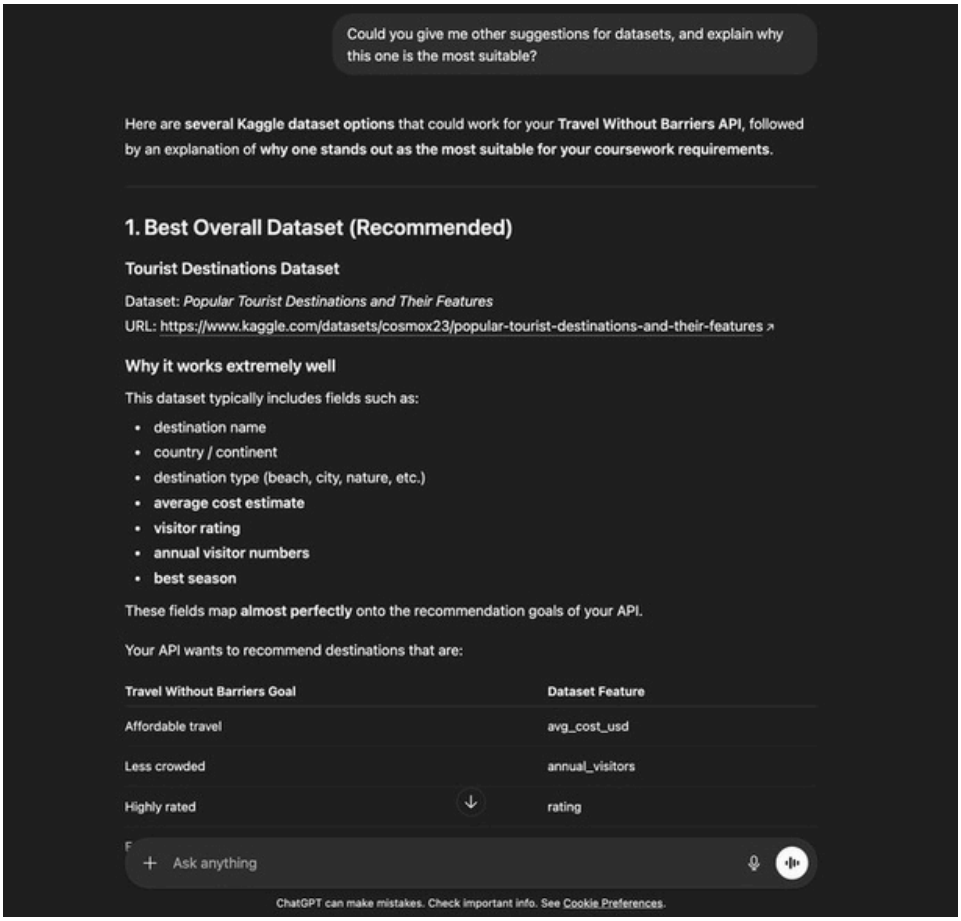
This log shows AI being used for data-engineering acceleration, in the case of implementing MCP. In the first screenshot, I refer to 'Phase 3', as GenAI had also been used to break down the completion of my tasks into phases of tasks during a previous conversation. ChatGPT gave me pieces of code to use, which I then reviewed, and implemented into my project, step-by-step, to make sure that we were aligned at each step for transparency and debugging purposes.

Figure A6. Debugging an SQL schema/query issue during development.



This screenshot is connected to Figure **A5** because it reflects what happened after generated code was implemented. AI was used as a debugging assistant to interpret error messages, for example in this case to explain the mismatch between intended and actual schema fields. Because I had requested for code to be generated step-by-step throughout implementation, this allowed for debugging to become more efficient, as ChatGPT had view of everything I was doing at each stage. The final fix still required manual verification and code changes.

Figure A7. Comparative dataset evaluation to justify why the chosen Kaggle source was the strongest fit.



Could you give me other suggestions for datasets, and explain why this one is the most suitable?

Here are **several Kaggle dataset options** that could work for your **Travel Without Barriers API**, followed by an explanation of **why one stands out as the most suitable for your coursework requirements.**

1. Best Overall Dataset (Recommended)

Tourist Destinations Dataset

Dataset: *Popular Tourist Destinations and Their Features*
URL: https://www.kaggle.com/datasets/cosmox23/popular-tourist-destinations-and-their-features

Why it works extremely well

This dataset typically includes fields such as:

- destination name
- country / continent
- destination type (beach, city, nature, etc.)
- **average cost estimate**
- **visitor rating**
- **annual visitor numbers**
- **best season**

These fields map **almost perfectly** onto the recommendation goals of your API.

Your API wants to recommend destinations that are:

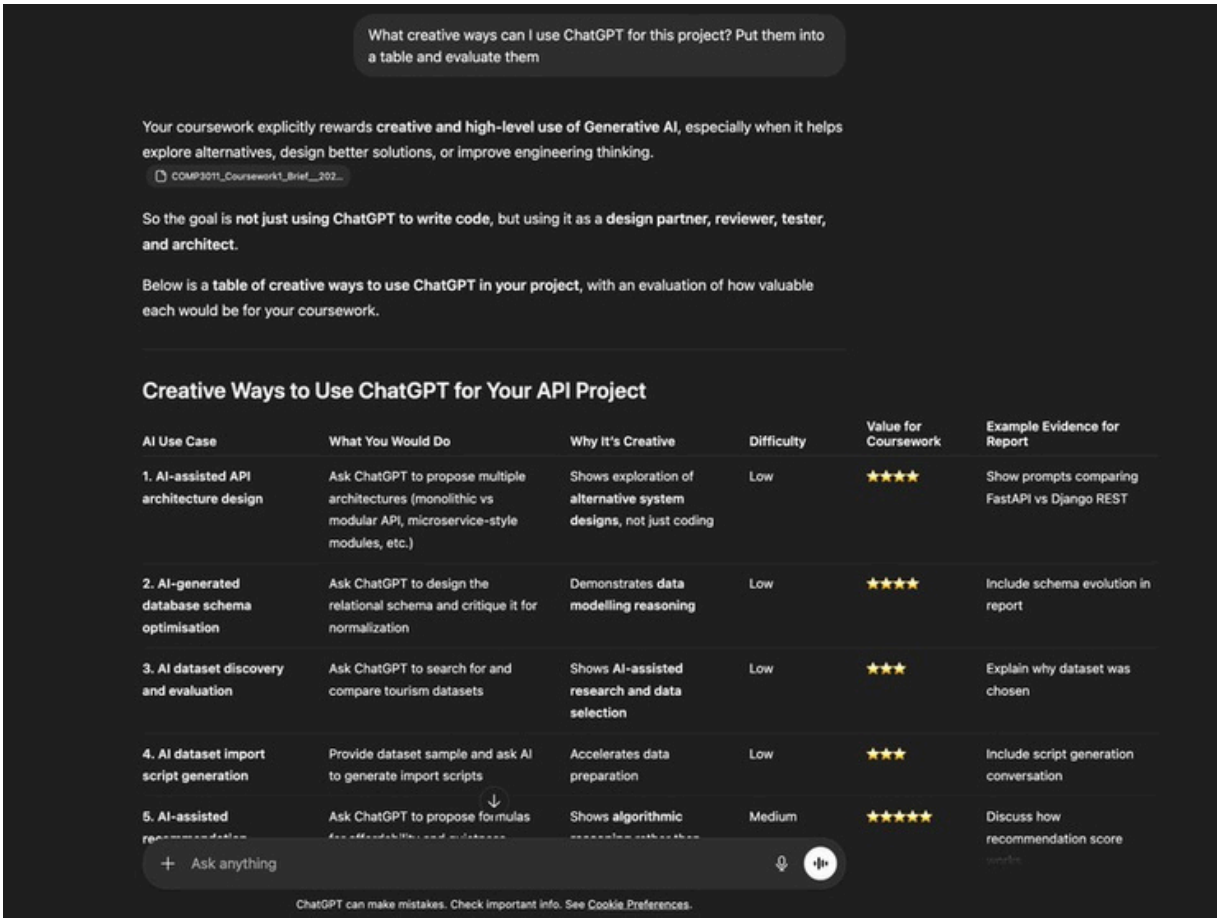
Travel Without Barriers Goal	Dataset Feature
Affordable travel	avg_cost_usd
Less crowded	annual_visitors
Highly rated	rating

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

This conversation provided a second layer of justification for dataset selection (shown in **A2**) by comparing alternatives and explaining why the chosen dataset mapped best to the project goal. This helped transform dataset selection from a casual choice into a defensible design decision.

Figure A8. Brainstorming creative ways to use ChatGPT within the coursework itself.



What creative ways can I use ChatGPT for this project? Put them into a table and evaluate them

Your coursework explicitly rewards **creative and high-level use of Generative AI**, especially when it helps explore alternatives, design better solutions, or improve engineering thinking.

COMP3011_Coursework1_Brief__202...

So the goal is **not just using ChatGPT to write code**, but using it as a **design partner, reviewer, tester, and architect**.

Below is a **table of creative ways to use ChatGPT in your project**, with an evaluation of how valuable each would be for your coursework.

Creative Ways to Use ChatGPT for Your API Project

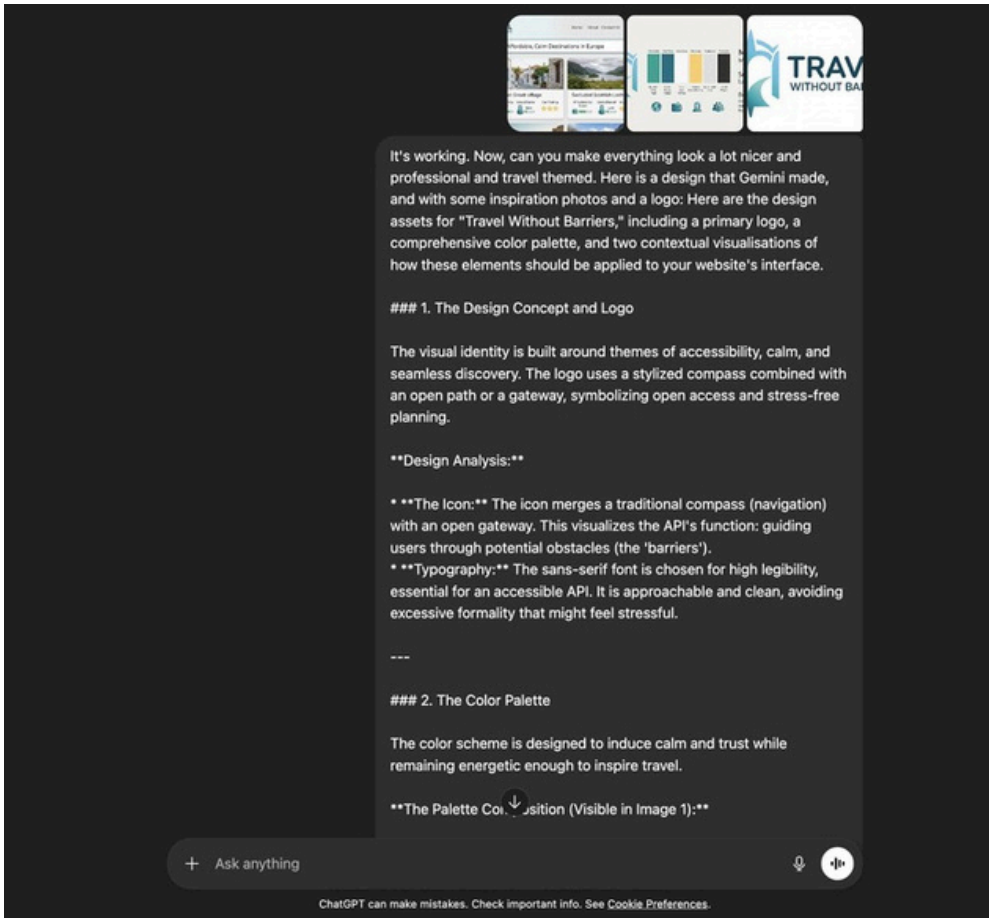
AI Use Case	What You Would Do	Why It's Creative	Difficulty	Value for Coursework	Example Evidence for Report
1. AI-assisted API architecture design	Ask ChatGPT to propose multiple architectures (monolithic vs modular API, microservice-style modules, etc.)	Shows exploration of **alternative system designs**, not just coding	Low	★★★★	Show prompts comparing FastAPI vs Django REST
2. AI-generated database schema optimisation	Ask ChatGPT to design the relational schema and critique it for normalization	Demonstrates **data modelling reasoning**	Low	★★★★	Include schema evolution in report
3. AI dataset discovery and evaluation	Ask ChatGPT to search for and compare tourism datasets	Shows **AI-assisted research and data selection**	Low	★★★	Explain why dataset was chosen
4. AI dataset import script generation	Provide dataset sample and ask AI to generate import scripts	Accelerates data preparation	Low	★★★	Include script generation conversation
5. AI-assisted	Ask ChatGPT to propose formulas	Shows **algorithmic**	Medium	★★★★★	Discuss how recommendation score

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

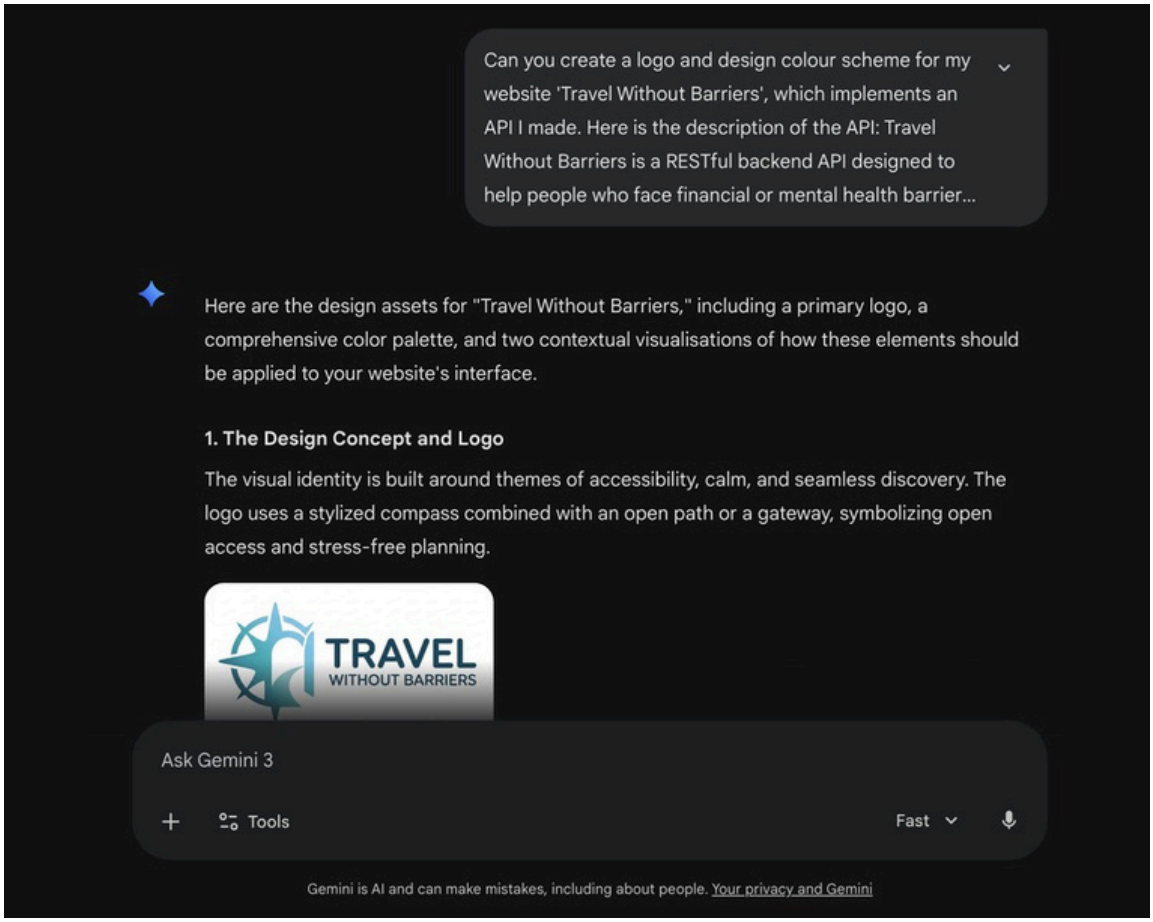
This screenshot shows reflective use of AI at the meta level: not only asking for outputs, but explicitly evaluating how AI could be used meaningfully within the project. This supported the later decision to include MCP, prompt engineering, debugging, dataset selection, and documentation assistance as deliberate AI-assisted workflows.

Figure A9. Gemini and ChatGPT being used to create brand and interface design guidance for the frontend of Travel Without Barriers.



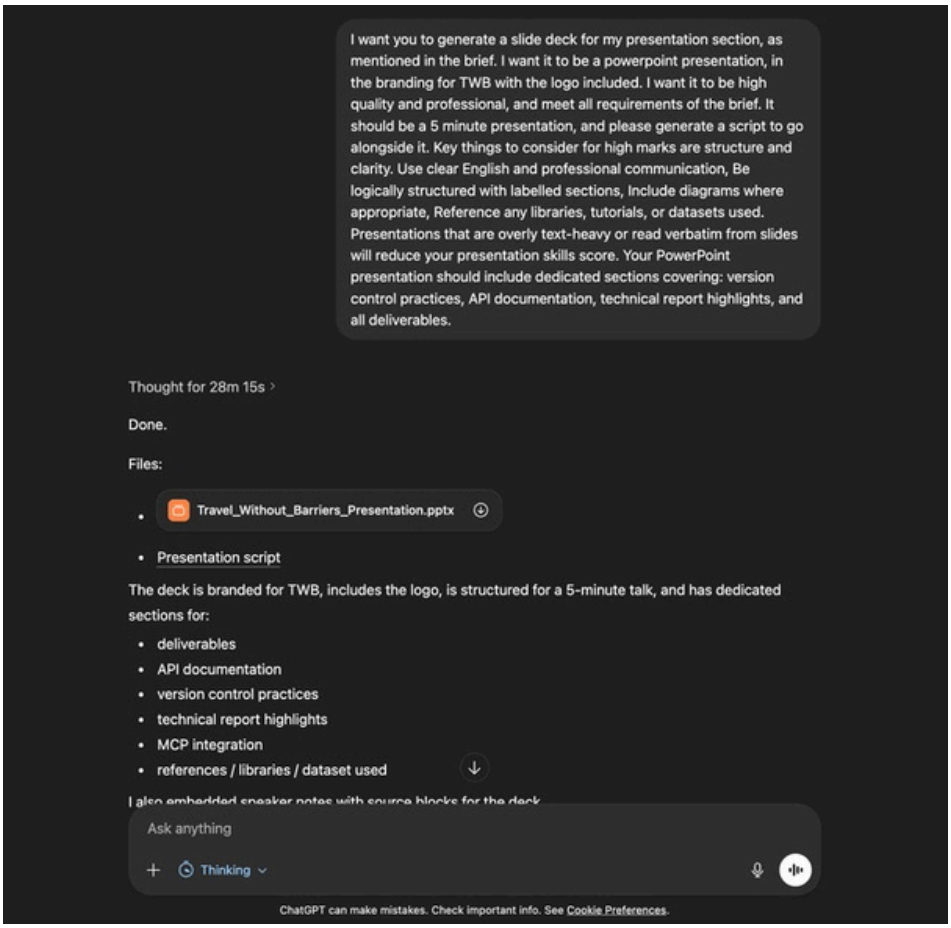
This shows me providing the branding generated by **Gemini** to **ChatGPT** and using it to design the frontend of the project. This screenshot shows Gemini being used as a secondary tool, specifically for visual identity and interface styling. The value here was comparative and creative rather than purely technical: it helped shape branding decisions, travel-themed presentation, and a calmer visual tone consistent with the product goal. The prompt in this screenshot was created using GenAI.

Figure A10. Gemini-generated branding assets, including the logo and colour direction used in the project.



This screenshot is connected to Figure **A9** and evidences the use of a second GenAI tool (Gemini) to generate the project's visual brand. The generated logo and palette informed the visual identity used in the frontend and documentation, showing that GenAI supported both engineering and communication outcomes. The description of the API used to inform Gemini of the API's functionality in the prompt was created by GenAI (ChatGPT).

Figure A11. GenAI used to create Powerpoint presentation and script



This screenshot shows ChatGPT being used to create a draft Powerpoint presentation, for the presentation section of the coursework. ChatGPT created the powerpoint, which was then reviewed by me, and adapted. ChatGPT created a draft script to use alongside the slides, and was also used as an assistant when I added the changes to the original draft.

Appendix B — Dataset Licence Notice

The dataset used in this project was sourced from Kaggle: Popular Tourist Destinations and Their Features. The following licence text was provided by the user for inclusion in the report and is reproduced here to document academic use and attribution expectations.

Released under MIT License

Copyright (c) 2013 Mark Otto.

Copyright (c) 2017 Andrew Fong.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies
of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.